

AUTOMATED REVIEW RATING SYSTEM

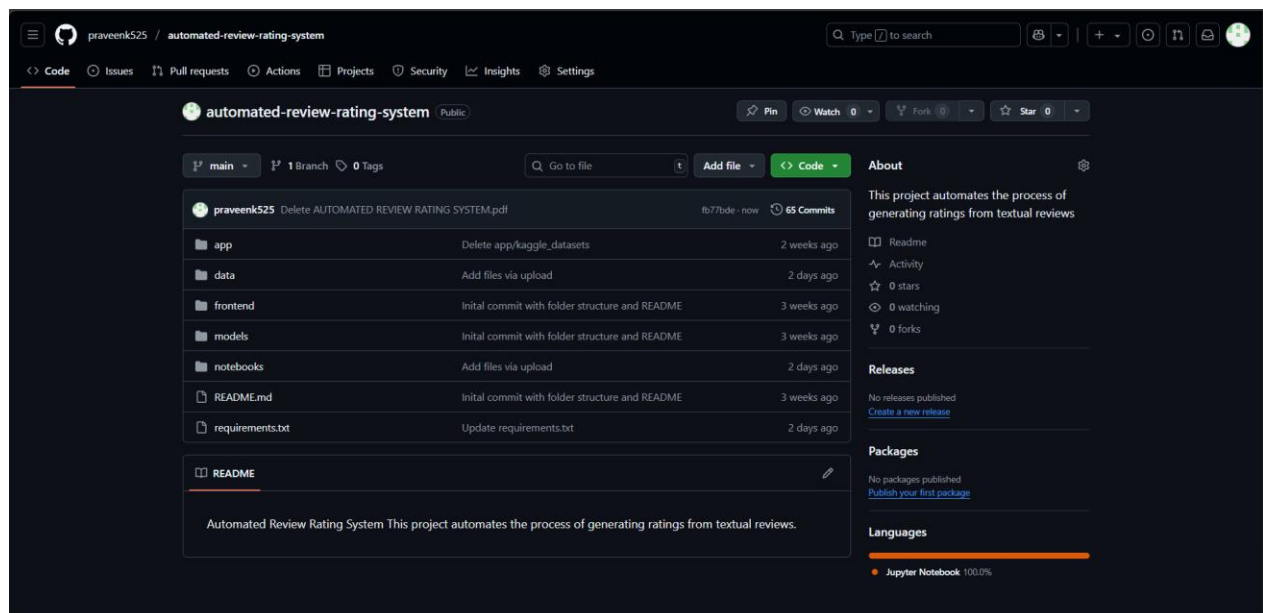
PROJECT OVERVIEW

This project automates the process of generating ratings from textual reviews. an AI-based system that automatically predicts the numerical rating (e.g., 1 to 5 stars) of a product, service, or business based solely on user-generated textual reviews. This aims to reduce rating manipulation, standardize feedback, and assist platforms in quickly gauging sentiment at scale.

ENVIRONMENT SETUP

- Installed python with necessary libraries such as numpy, pandas, matplotlib, scikit-learn, seaborn, nltk
- Colab

GITHUB PROJECT REPOSITORY



DATA COLLECTION

Data was collected from Kaggle and google datasets.

The project uses 10 diverse datasets across industries to ensure model generalization:

- Women's Clothing Reviews
- TripAdvisor Hotel Reviews
- Laptop Product Reviews
- IMDB Movie Reviews
- Amazon Product Reviews (including various categories)
- Plus 5 other domain-specific review datasets (e.g., food, electronics, services, etc.)

Each dataset contains:

- Review text
- Corresponding user rating (1–5 stars)
- Optional fields: title, product ID, date, reviewer info

S.No.	Dataset Name	Domain	Source	Approx. Size	Columns	Links
1	Recipe reviews	Food & Hospitality	Kaggle	~18182	15	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
2	Reviews& ratings	E-commerce	Kaggle	~1535	8	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
3	Walmart	E-commerce	Kaggle	~300	6	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
4	Womens clothing	E-commerce	Kaggle	~23486	11	https://github.com/praveenk525/automated-review-rating-system/blob/main/data/kaggle_datasets/Womens%20Clothing%20E-Commerce%20Reviews.csv
5	Amazon reviews	E-commerce	Kaggle	~4915	12	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets

6	imdb_tvshoows	Movies & Tv	Google	~8319	5	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
7	Laptops	Electronics	Kaggle	~24113	7	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets
8	review_details	E-commerce	Kaggle	~2501	32	https://github.com/praveenk525/automated-review-rating-system/tree/main/data/kaggle_datasets

All datasets are merged into one

Merged_dataset link=https://github.com/praveenk525/automated-review-rating-system/blob/main/data/merged_datasets/Merged_dataset.csv

DATA PREPROCESSING

Preprocessing is the first and most important step in any machine learning or data science project. It involves cleaning, transforming, and organizing raw data into a format that your machine learning model can understand and learn from effectively.

Load and Inspect Datasets

- Load all 10 datasets (e.g., Women's Clothing, TripAdvisor, etc.)
- Inspect structure: check column names, data types, sample records
- Verify presence of review and rating columns `pd.read_csv()` → Load CSV files into

a DataFrame `pd.read_excel()` → For Excel files

View first few rows

`df1.head()`

View last few rows

`df1.tail()`

```
# Check column names

df1.columns

# Get DataFrame shape (rows, columns) df1.shape

# Check column data types

df1.dtypes # Full summary

info df1.info()

# Summary statistics (for numeric columns) df1.describe()
```

Remove Unnecessary Columns

- Drop columns not required for prediction, such as:
 - user_id, product_id, timestamp, title, image_url, etc.
- Keep only relevant columns: usually review_text and rating

Remove Null or Missing Values

- Remove rows with null values in review or rating

```
#Check for missing values in each column

print(df.isnull().sum())

# removing null values

df.dropna(inplace=True)

# Remove rows where review_text or rating is missing df.dropna(subset=['review_text',
'rating'], inplace=True)
```

Remove Duplicates and Conflicting Reviews

- **Exact duplicates:** same review and rating — keep one

- **Conflicting reviews:** same review text, different ratings — remove all #check

duplicates print(df.duplicated().sum()) # Remove all duplicate rows

df.drop_duplicates(inplace=True)

Clean Text Data

- Convert text to lowercase
- Remove:
 - URLs
 - Punctuation
 - Numbers and special characters
 - Extra spaces

```
import re

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"^\w\s", "", text)
    text = re.sub(r"\d+", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df.loc[:, "review"] = df["review"].apply(clean_text)
```

Stopwords Removal

Stopwords are common words in a language (like *"the"*, *"is"*, *"at"*, *"which"*, *"and"*) that do not add significant meaning to text and are usually removed during text preprocessing in NLP tasks.

Why Remove Stopwords?

- They don't help in understanding sentiment or meaning.
- They inflate feature space and slow down training.
- Removing them improves model accuracy and efficiency.

```

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download('stopwords')
nltk.download('punkt')

stop_words = set(stopwords.words('english'))

def remove_stopwords(text):
    words = word_tokenize(str(text)) # tokenize text
    filtered = [word for word in words if word.lower() not in stop_words]
    return ' '.join(filtered)

df['review']=df['review'].apply(remove_stopwords)

```

What is Lemmatization?

Lemmatization is the process of reducing a word to its root form (lemma) based on its dictionary meaning and part of speech.

Example:

- "running" → "run"
- "better" → "good"
- "was" → "be"

Lemmatization uses a vocabulary (dictionary) and morphological analysis to correctly identify the base form of a word.

What is Stemming?

Stemming is a simpler, rule-based process that chops off prefixes or suffixes to get to the root word. It doesn't care about grammar or word meaning.

Example:

- "running" → "run"
- "flying" → "fli"
- "studies" → "studi"

Why Lemmatization is Better (in most ML tasks):

- Returns actual words (important for readability and clarity).
- More grammatically correct results.

- Handles irregular words better (e.g., "better" → "good").
- Reduces noise compared to stemming.

```

nltk.download('wordnet')
nltk.download('omw-1.4')
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
lemmatizer = WordNetLemmatizer()
def lemmatize_text(text):
    words = word_tokenize(text)
    lemmatized = [lemmatizer.lemmatize(word) for word in words]
    return ' '.join(lemmatized)
df['review']=df['review'].apply(lemmatize_text)

```

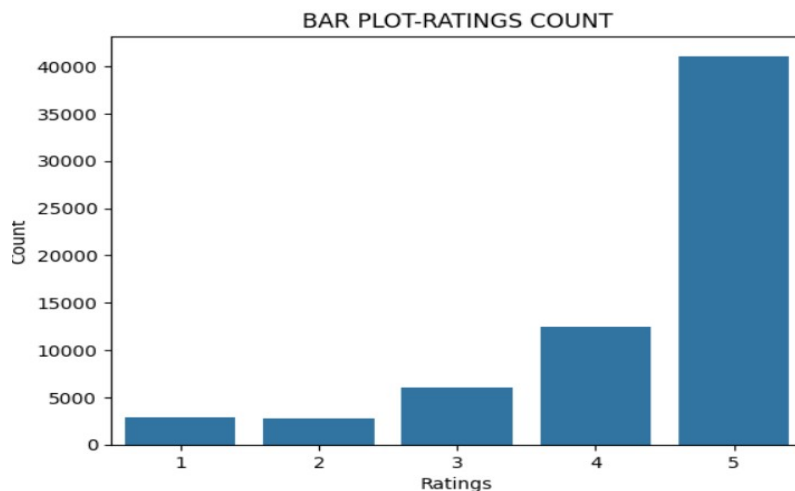
DATA VISUALIZATION

Data Visualization is the process of representing data and information visually using charts, graphs, and other visual elements (like maps or word clouds) to help people understand patterns, trends, and insights in data.

BAR CHART

A bar chart represents categorical data with rectangular bars. The length of each bar is proportional to the value it represents.

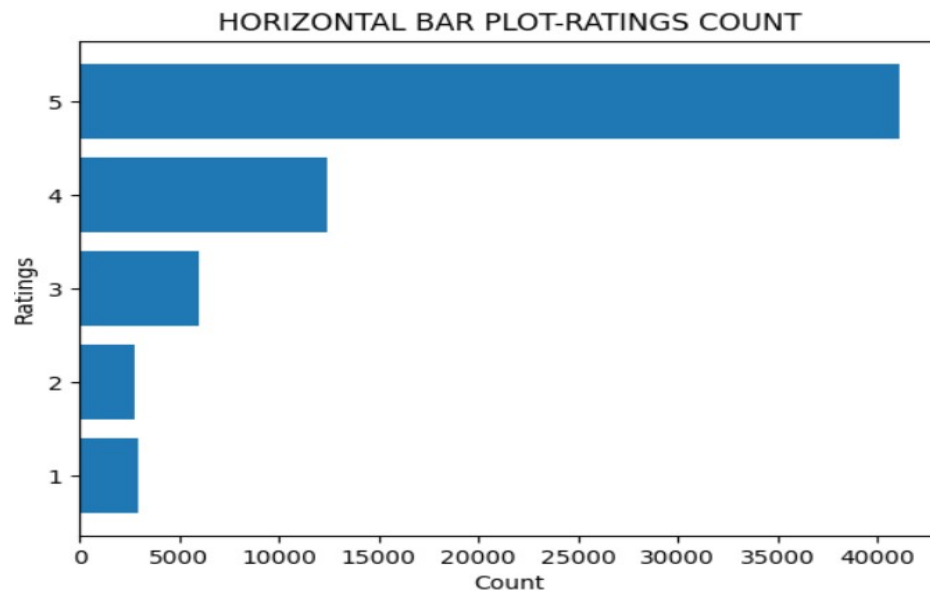
- Use: To show the number of reviews in each star rating (e.g., how many 1-star, 2-star, etc.).
- Number of reviews per rating (1 to 5).



Horizontal Bar Chart (HBar)

A horizontal bar chart is like a bar chart, but the bars extend horizontally instead of vertically.

- To show average values like review length for each rating in a clean, readable format.



Pie Chart

A pie chart is a circular chart divided into slices to illustrate proportions of categories.

- To show the percentage of each rating (how much of the data is 5-star, 4-star, etc.).

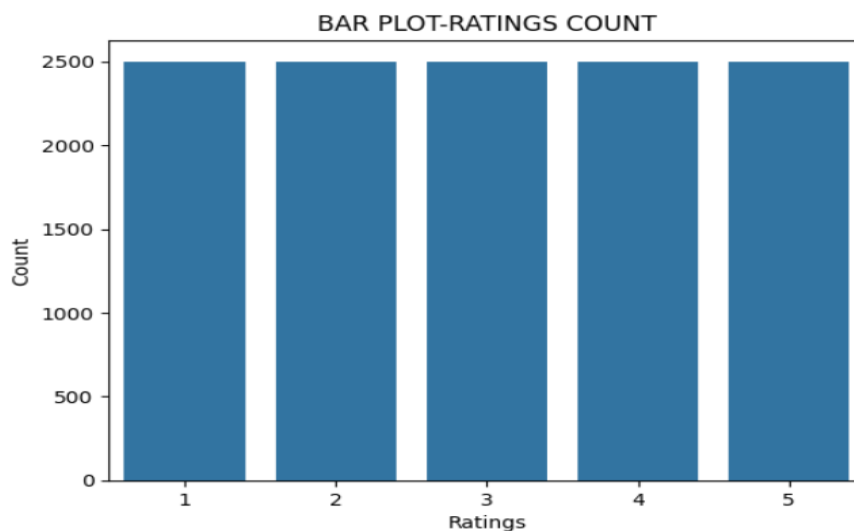
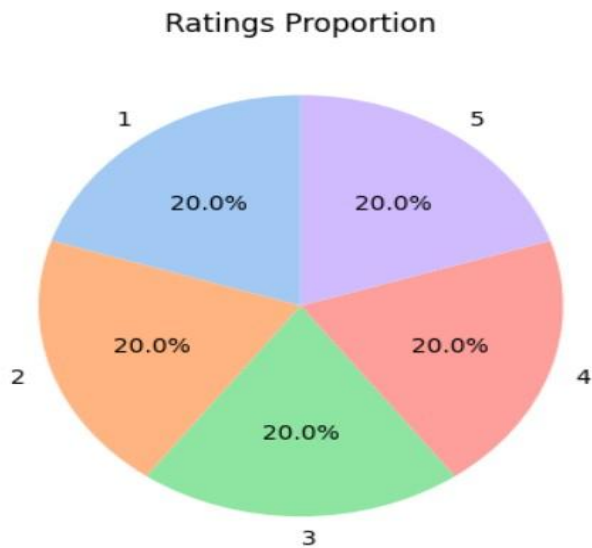


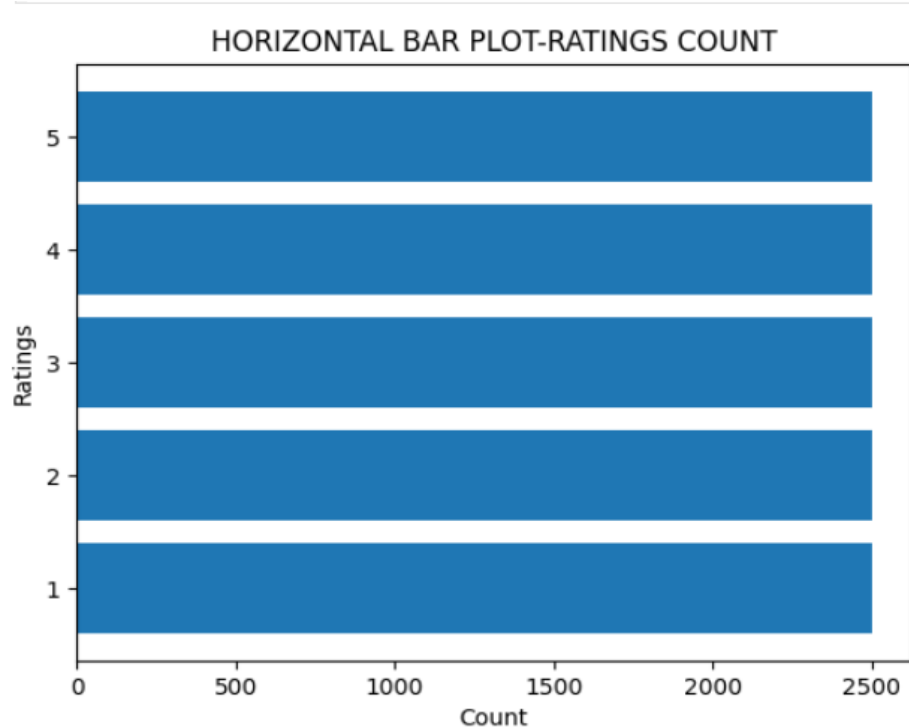
BALANCED DATASET

A balanced dataset is one in which all classes or categories have an equal (or nearly equal) number of samples. This ensures that the machine learning model learns from all classes equally and doesn't become biased.

Balanced dataset link=https://github.com/praveenk525/automated-review-ratingsystem/blob/main/data/balance_imbalance_datsets/Balanced_dataset.csv

Balanced dataset with 2000 reviews and ratings





SAMPLES:

Showing 5 sample review(s) for Rating 1:

1. ran several test computer phone wrote mb read depeneding mood bought replace generic gb class card lost dang sm all dropped could find card actual wrote mb read mb went sandisk phone came class performs like class waste time ref unded item
2. disappointed recipe agree tilda dry flavor expecting orangey tasting cake could taste orange made grandson age car e love everything make easy please please throw away
3. quality display bad got effective speaker microsoft installed waste money replacing also dont buyy
4. dress cute went store try although knew dress would fit could get zipper mom sale lady tried get zipper would mov e past ribbon wish could bought dress cant without working zipper tried size every dress store size none worked pleas e fix zipper problem dress
5. impressed everything season everything perfect bad worse unwatchable gon na watch keep checking episode rating figure managed save dont care whats next getting corny pretentious extent started dislike elisabeth moss three season enough clue really storyline nothing look promising thank first two season okay rest elisabeth moss find another show

Showing 5 sample review(s) for Rating 2:

1. really wanted love dress absolutely beautiful color design detail however whether high neckline sleeve nothing ma de look kinda frumpy though slender chest maybe problem
2. super soft fabric however short stride ordered xl normally wear return

3. purchased dress bowback shift peach loved nautical theme pattern recurve dress found stiff boxy petite shortwaisted person top portion stress hung body awkwardly bra strap showing back dress needed pulled in properly sit frame returned dress

4. love love love print retailer sale think regular price way much dress dress isn't even silk better material got disappointed ordered usual size ballooned felt like wearing maternity dress plus ruffle top thought sweet feature felt really awkward like grazing neck chin annoying way don't even want

5. neck big sweater couldn't make lay right shape sweater odd really wide upper torso taper near waist look depicted model sweater lovely color liked two-tone overall fit weird material soft felt really nice oddly shaped crazy big neck

Showing 5 sample review(s) for Rating 3:

1. I'm going responsible thing establish bias right but loved first game much father day wife managed buy p copy last u part ii family time sequestered away played game start finish last u favorite franchise last u part ii favorite game din a favorite character isabella merced favorite currently active actress mind torn reaction show true fan would appalled change casting I'm torn absolutely love show find change strength rather weakness already played game u many many time audience never attachment expectation different change show accommodate also narratively interesting important need remember different perspective different telling different approach familiar story different myth changed time different comic give different take character make MCU work could gush much show important remind u show story given another way enjoy something great though complaint physicality Abby's character actually important everyone disservice choice shame

2. best TV serious ever see TV game throne please Netflix wait season

3. slow expensive better option

4. perfect laptop except heating issue

5. ordered pink size XL color pretty problem material thin tight ran least size smaller shown photo online

Showing 5 sample review(s) for Rating 4:

1. good laptop performance great display bit mesh overall great

2. best laptop segment OK go

3. automatically knocked one star every brand offer size X many also offer XX also care u size XL well ordered size large course show every bump love handle back recreated look local retailer window showed long unbuttoned cotton weave dress crochet dress closest dress pictured online saw mabli maxi dress imagine opened button blue

4. dress fit great warm winter month

5. seems working fine bought gift family member they're using video recording

Showing 5 sample review(s) for Rating 5:

1. love soup fast economical make many time winter month

2. ordered top size heard Mara Hoffman run small size chart site would order size bust however bra size C size fit there fore might also fit B back adjustable even stay without halter highly unusual negative really disappointed pattern quite symmetrical expect brand

3. purchased sandisk ultra gb microsdxc class uhs memory card adapter use samsung galaxy note android apparently phone problem recognition large sd card make possible save lot photo video feeling even possible get gb version work well phone make sure fully compatible reformat using phone sd card phone able detect utilize full capacity problem overall extremely satisfied product

4. love combination three product buy together work well together fragrance smell nice

5. bought new grandson daughter love help putting sleep

Train-Test Split

What is Train-Test Split?

In machine learning, we split the dataset into:

- **Training Set** → Used to train the model (e.g., 80% of data)
- **Testing Set** → Used to evaluate the model's performance (e.g., 20% of data)

This helps ensure the model can generalize to unseen data.

TRAIN_TEST_SPLIT

```
from sklearn.model_selection import train_test_split

# Suppose 'df' has columns: 'review' and 'rating'

# Features (X) and Labels (y)
X = df['review']
y = df['rating']

# Perform stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

# Optionally combine X and y back into DataFrames
train_df = pd.DataFrame({'review': X_train, 'rating': y_train})
test_df = pd.DataFrame({'review': X_test, 'rating': y_test})
```

Why Use stratify?

If the dataset is imbalanced (e.g., more 5-star reviews than 1-star), a random split could cause:

- Training set to have fewer examples of rare classes.
- Testing set to not represent all classes equally.

Stratification ensures:

- The class distribution (percentage of each rating) is the same in both train and test sets.

TF-IDF Vectorization

What is TF-IDF?

TF-IDF (Term Frequency – Inverse Document Frequency) is a feature extraction technique used in NLP to convert text into numerical vectors.

It assigns higher weight to important words in a document and lower weight to common words.

Why Use TF-IDF?

- Removes bias toward frequent but unimportant words (e.g., "the", "and").
- Captures the importance of words in relation to all reviews.
- Works well for text classification tasks like review rating prediction.

How It Works

1. TF (Term Frequency)

Measures how often a word appears in a document.

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

2. IDF (Inverse Document Frequency)

Measures how important a word is across all documents.

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term}}\right)$$

3. TF-IDF SCORE

$$TF-IDF = TF * IDF$$

TF-IDF VECTORIZER

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=5000, stop_words='english')
X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)
```